

ME3302: Automation in Manufacturing

Final Report

Group 9

ME22B208 Trisha Wadhwani

ME22B223 Shyam Kumar

ME22B224 Mayank Chandak

ME22B227 Mahendra Kurup

April 2025

Project Repository:

[Link](#)

Contents

1	Problem Statement	2
2	Our Approach	2
2.1	Why Reinforcement Learning?	2
3	Custom Environment: Multi-Agent Warehouse Simulation	3
3.1	Environment Architecture	3
3.2	State Representation	4
3.3	Action Space	4
3.4	Observation Models	4
3.4.1	Centralized Observation	5
3.4.2	Decentralized Observation	5
3.4.3	Decentralised Partial Observability	5
3.5	Dynamics and Request Generation	5
3.6	Reward Structure	6
3.7	Environment Dynamics and Episode Termination	6
3.8	Visualization Capabilities	8
3.9	Environmental Complexity and Challenges	9
4	Algorithms Employed	9
4.1	Proximity-based Task Allocation Strategy	9
4.2	Centralized Training with Decentralized Execution	11
4.2.1	Code Implementation	11
4.2.2	Shortcomings	14
4.3	Centralized PPO	15
4.3.1	Shortcomings	16
4.4	Hierarchical Reinforcement Learning (HRL)	16
4.4.1	Shortcomings	16
5	Evaluation Metrics	17
5.1	Task and Operational Efficiency	17
5.2	Path Planning and Motion Efficiency	17
5.3	Safety and Collision Avoidance	18
6	Future Improvements	18

1 Problem Statement

In modern logistics and supply chain systems, the demand for efficient, scalable, and intelligent warehouse automation has grown significantly. Traditional warehouse management systems often rely on pre-defined rules and static path planning, which can lead to inefficiencies in dynamic environments. This project aims to develop an intelligent, multi-agent warehouse management system using Autonomous Guided Vehicles (AGVs) capable of performing efficient **path planning** and **task allocation** in a dynamic, multi-location warehouse setting.

The system addresses two fundamental challenges:

1. **Path Planning:** Ensuring collision-free and optimal path computation for multiple AGVs navigating a shared space with dynamically changing obstacles (e.g., moving agents, blocked shelves). The goal is to minimize travel time and maximize throughput.
2. **Task Allocation:** Efficiently assigning storage and retrieval tasks to AGVs in real-time based on their current state (position, load status, availability) and warehouse conditions (shelf availability, urgency of tasks).

To simulate and evaluate solutions, we designed a *multi-agent grid world environment* that represents a realistic abstraction of a warehouse with multiple shelves and delivery/pickup zones. Each AGV operates within this environment and must complete tasks collaboratively under constraints such as limited space and potential task conflicts.

2 Our Approach

We tested various reinforcement learning (RL) and heuristic-based approaches for solving this problem, including:

- **Centralized Training with Decentralized Execution (CTDE)** frameworks for scalable coordination among agents.
- **Centralized policy learning** methods such as PPO and SAC for joint strategy development.
- **Hierarchical Reinforcement Learning (HRL)** to separate high-level task assignment from low-level navigation.
- **Heuristic methods** like A* for baseline comparison and integration with hybrid planning approaches.

Our goal is to benchmark and identify algorithms that provide the best trade-offs in terms of task efficiency, path optimality, and adaptability to warehouse dynamics.

2.1 Why Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning paradigm where agents learn to make sequential decisions by interacting with an environment. At each timestep, an agent observes the current state, takes an action, receives a reward, and transitions to a new state. Over time, the agent learns an optimal policy that maximizes cumulative reward through trial and error.

Formally, the RL problem is often modeled as a Markov Decision Process (MDP), defined by a tuple (S, A, P, R, γ) where:

- S is the set of states,
- A is the set of actions,
- $P(s'|s, a)$ defines the transition probabilities,
- $R(s, a)$ is the reward function,
- $\gamma \in [0, 1)$ is the discount factor.

In the context of autonomous warehouse systems, RL offers several advantages over traditional rule-based or heuristic approaches:

- **Adaptability to Dynamics:** RL agents can adapt to changing warehouse layouts, blocked paths, and varying task priorities without requiring explicit reprogramming.
- **Scalable Multi-Agent Coordination:** Through frameworks like CTDE and shared policies, RL enables decentralized coordination among multiple AGVs with shared objectives and partial observability.
- **Learning from Interaction:** Unlike precomputed path planning algorithms (e.g., A*), RL agents learn from actual environment interaction and can improve over time with more experience.
- **End-to-End Optimization:** RL enables joint optimization of navigation and task selection, as opposed to disjoint modules for scheduling and routing in classical methods.
- **Reward Shaping for Custom Objectives:** Designers can define complex reward structures to encourage desired behaviors like traffic minimization, shelf priority, energy efficiency, or urgency handling.

While classical methods like A* or rule-based task allocation are easier to implement and interpret, they often lack the flexibility and robustness needed for dynamic, large-scale, and partially observable environments. RL provides a unified learning framework that improves with more data and supports both continuous control and discrete decision-making—making it highly suitable for intelligent warehouse automation.

3 Custom Environment: Multi-Agent Warehouse Simulation

This section provides a comprehensive analysis of our custom-designed warehouse environment for multi-agent reinforcement learning. The environment simulates a warehouse logistics system where multiple agents must collaborate to pick up packages from shelves and deliver them to designated destinations, mirroring real-world warehouse automation challenges.

3.1 Environment Architecture

The warehouse environment is implemented as a custom Gymnasium environment, inheriting from the `gym.Env` class. The core environment is defined in the `WarehouseEnv` class, which encapsulates the state space, action space, dynamics, and reward structure necessary for reinforcement learning.

```

1 class WarehouseEnv(gym.Env):
2     def __init__(self, grid_size=(15, 10), num_agents=2, num_shelves=10,
3                 shelf_spacing=2, shelf_length=4, poisson_lambda=0.1,
4                 observation_type='centralized', normalize=True,
5                 max_requests=5, goal_deliveries=10):
6         super().__init__()
7         # Environment configuration parameters
8         self.grid_size = grid_size
9         self.num_agents = num_agents
10        self.num_shelves = num_shelves
11        self.shelf_spacing = shelf_spacing
12        self.shelf_length = shelf_length
13        self.poisson_lambda = poisson_lambda
14        self.observation_type = observation_type
15        self.normalize = normalize
16        self.max_requests = max_requests
17        self.goal_deliveries = goal_deliveries
18        # ...

```

The environment is highly configurable with parameters controlling:

- `grid_size`: Dimensions of the warehouse grid (rows, columns)
- `num_agents`: Number of robots operating in the warehouse

- `num_shelves`: Number of storage shelves in the warehouse
- `shelf_spacing` and `shelf_length`: Physical layout parameters
- `poisson_lambda`: Rate parameter for Poisson process generating package requests
- `observation_type`: Determines agent observation model (centralized, decentralized, or partial observability)
- `max_requests`: Maximum number of active package requests
- `goal_deliveries`: Target number of deliveries to complete the episode

3.2 State Representation

The warehouse state is primarily represented through a 2D grid where each cell can contain:

- Empty space (0)
- Shelf (1)
- Destination (2)
- Agent (3)
- Package (4)

This grid representation is maintained in the `self.grid` attribute, a NumPy array of shape `grid_size`. Additional state information includes agent positions, orientations, package carrying status, and active requests:

```

1  # State variables
2  self.grid = np.zeros(self.grid_size, dtype=int)
3  self.agent_positions = []           # List of (x, y) tuples
4  self.agent_orientations = []       # 0: up, 1: right, 2: down, 3: left
5  self.agent_carrying = [None for _ in range(num_agents)]
6  self.requests = []                # (request_id, src, dest, picked)

```

The environment dynamically updates this state representation during initialization and after each step through methods like `_place_shelves()`, `_place_destinations()`, and `_update_grid()`.

3.3 Action Space

The action space is defined as a `MultiDiscrete` space, allowing each agent to perform one of seven possible actions:

```

1  self.action_space = spaces.MultiDiscrete([7] * num_agents)
2  # 0: Move forward
3  # 1: Move backward
4  # 2: Rotate left
5  # 3: Rotate right
6  # 4: Pick box
7  # 5: Place box
8  # 6: Idle

```

This action space provides robots with mobility (movement and rotation) and manipulation capabilities (picking and placing packages), forming a rich decision space for learning complex behaviors.

3.4 Observation Models

A distinctive feature of this environment is its support for multiple observation models, catering to different multi-agent reinforcement learning paradigms:

3.4.1 Centralized Observation

In centralized mode, the environment provides a single comprehensive observation of the entire state:

```
1  if observation_type == 'centralized':
2      # Grid + agents info + requests info
3      obs_size = (grid_size[0] * grid_size[1]) + (num_agents * 4) + (self.max_requests * 6)
4      self.observation_space = spaces.Box(low=-1, high=max(grid_size[0], grid_size[1], 3, self.max_requests),
5      shape=(obs_size,), dtype=np.float64)
```

This observation includes the complete grid state, information about all agents (position, orientation, carrying status), and all active requests. This model is appropriate for centralized training approaches where a single policy controls all agents.

3.4.2 Decentralized Observation

The decentralized observation mode provides each agent with its own perspective:

```
1  elif observation_type == 'decentralized':
2      # Grid + agent info + requests info
3      obs_size = (grid_size[0] * grid_size[1]) + 4 + (self.max_requests * 6)
4      self.observation_space = spaces.Box(low=-1, high=max(grid_size[0], grid_size[1], 3, self.max_requests),
5      shape=(num_agents, obs_size), dtype=np.int8)
```

Each agent still observes the entire grid but has agent-specific information about its own position, orientation, and carrying status. This model supports decentralized training approaches.

3.4.3 Decentralised Partial Observability

For more realistic scenarios, the environment also supports partial observability:

```
1  elif observation_type == 'decentralized_partial_observability':
2      # 5x5 partial grid + agent info + requests info
3      obs_size = (25 + 4 + (self.max_requests * 6))
4      self.observation_space = spaces.Box(low=-1, high=max(grid_size[0], grid_size[1], 3, self.max_requests),
5      shape=(num_agents, obs_size), dtype=np.int8)
```

In this mode, each agent only sees a 5×5 grid centered on its position, restricting its perception to local information. This model introduces additional complexity and realism to the learning task.

3.5 Dynamics and Request Generation

The environment implements realistic warehouse dynamics with packages appearing according to a Poisson process:

```
1  if len(self.requests) >= self.max_requests:
2      return
3  if self.deliveries_completed + len(self.requests) >= self.goal_deliveries:
4      return
5  if np.random.rand() < self.poisson_lambda:
6      shelf_cells = list(zip(*np.where(self.grid == 1)))
7      src = random.choice(shelf_cells)
8      dest = random.choice(self.destinations)
9      # Add request ID to the request tuple
10     self.requests.append((self.next_request_id, src, dest, False))
11     self.next_request_id += 1
```

This method dynamically generates new package requests. Package sources are randomly selected from shelf locations, and destinations are chosen from pre-defined destination points along the warehouse perimeter.

3.6 Reward Structure

The environment implements a sophisticated reward structure that incentivizes efficient package delivery while penalizing undesirable behaviors:

```
1  # Reward components in _apply_action method
2  if action == 0: # move forward
3      if 0 <= fx < self.grid_size[0] and 0 <= fy < self.grid_size[1] \
4          and self.grid[fx, fy] == 0:
5          self.agent_positions[agent_id] = (fx, fy)
6          reward += -1 # Small penalty for movement
7          new_pos = (fx, fy)
8      elif not (0 <= fx < self.grid_size[0] and 0 <= fy < self.grid_size[1]):
9          reward += -10 # Larger penalty for invalid actions
10     # ...
11
12 elif action == 4: # pick
13     if self.agent_carrying[agent_id] is None:
14         for i, (req_id, src, _, picked) in enumerate(self.requests):
15             if src == (fx, fy) and not picked:
16                 self.agent_carrying[agent_id] = req_id
17                 self.requests[i] = (req_id, src, self.requests[i][2], True)
18                 reward += 50 # Substantial reward for picking up packages
19                 break
```

Key reward components include:

- Small negative rewards (−1) for basic actions (movement, rotation, idling)
- Large negative rewards (−10) for illegal actions (collisions, boundary violations)
- Positive rewards for successful pickups (+50)
- Large positive rewards for deliveries (+100)
- Distance-based rewards (+3/ − 3) when carrying packages toward/away from destinations
- Team rewards (+4) when any agent completes a delivery

This multi-factor reward design creates a challenging learning problem that requires agents to balance immediate rewards (avoiding penalties) with delayed rewards (completing deliveries).

3.7 Environment Dynamics and Episode Termination

The environment’s step function orchestrates the state transitions, action execution, and reward calculations:

```
1  def step(self, actions):
2      self._poisson_requests()
3      rewards = []
4      prev_num_requests = len(self.requests)
5
6      for i, action in enumerate(actions):
7          rewards.append(self._apply_action(i, action))
8
9      self._update_grid()
10
11     # Collision penalties
12     for i, pos in enumerate(self.agent_positions):
13         if self.agent_positions.count(pos) > 1:
14             rewards[i] += -20
```

```

15
16     # Track deliveries completed and reward all agents
17     if prev_num_requests > len(self.requests):
18         self.deliveries_completed += prev_num_requests - len(self.requests)
19         for i in range(self.num_agents):
20             rewards[i] += 4
21
22     self.steps_taken += 1
23     terminated = [False] * self.num_agents
24     if self.deliveries_completed == self.goal_deliveries:
25         terminated = [True] * self.num_agents
26     truncated = True if self.steps_taken >= 1500 else False
27     # ...

```

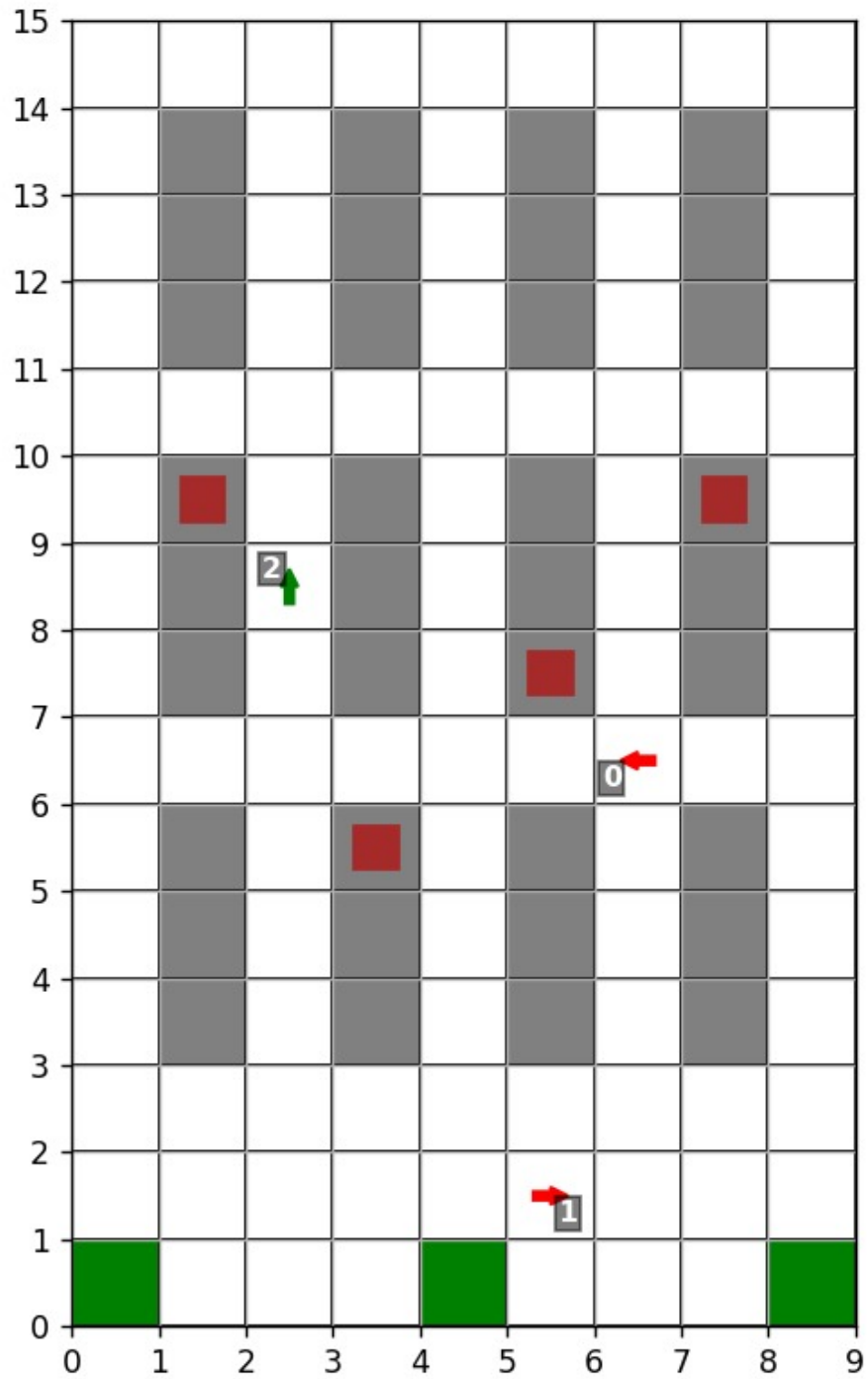
Episodes terminate under two conditions:

- Successfully completing the target number of deliveries (`goal_deliveries`)
- Exceeding the maximum number of steps (1500)

This design creates episodes with clear success criteria while avoiding indefinite execution.

3.8 Visualization Capabilities

The environment includes sophisticated rendering capabilities using Matplotlib:



The visualization shows:

- Warehouse grid layout with shelves and destinations
- Agent positions and orientations (represented as arrows)

- Package source(Red block) and destination locations(Specific green block for each request)
- Package carrying status (indicated by agent color, Green for a loaded agent and Red for free agent)

This visualization component is valuable for debugging, monitoring agent behavior, and gaining insights into learned policies.

3.9 Environmental Complexity and Challenges

Several factors combine to make this warehouse environment a challenging multi-agent reinforcement learning problem:

1. **Partial observability:** When using the partial observability mode, agents must make decisions with incomplete information about the environment state.
2. **Stochastic dynamics:** The Poisson process for generating package requests introduces unpredictability and requires agents to adapt to changing demand patterns.
3. **Multi-agent coordination:** Agents must avoid collisions and ideally coordinate to efficiently handle package requests.
4. **Long-horizon rewards:** The delay between picking up packages and receiving the large delivery reward creates a challenging temporal credit assignment problem.
5. **Sparse rewards:** Many actions result in small negative rewards, with significant positive rewards only for successful pickups and deliveries.

Key strengths of this implementation include:

- Comprehensive state representation with rich dynamics
- Flexible observation models supporting different learning paradigms
- Detailed reward structure that encourages desired behaviors
- Realistic package request generation via Poisson process
- Built-in visualization capabilities for analysis and debugging

This environment serves as an ideal platform for researching multi-agent coordination, partial observability challenges, and reinforcement learning approaches to logistical optimization problems.

4 Algorithms Employed

4.1 Proximity-based Task Allocation Strategy

We implemented a basic proximity-based task allocation strategy for coordinating multiple delivery robots (agents) in a grid-based warehouse environment. The core idea is to assign each new pickup request to the **nearest available agent** based on *Manhattan distance*, and then plan a **collision-free path** for that agent using the A* search algorithm.

Task Allocation Strategy

When a new pickup request is made (i.e., a delivery task appears at a particular grid location), the system performs the following:

- **Agent Availability Check:**

We maintain a list `agent_busy` indicating whether each agent is currently handling a task. Only idle (free) agents are considered for assignment.

- **Proximity-based Assignment:**

For each available agent, we calculate the Manhattan distance between the agent's current position and the pickup location. The task is assigned to the agent with the smallest distance, using the `find_nearest_free_agent()` function.

Path Planning with A*

Once an agent is assigned a task, it needs to navigate to the pickup location:

- **Path Computation:**

We use the `a_star()` function to compute the optimal path from the agent's current position to the task location on the grid. A* ensures that the path is both *efficient* (shortest possible) and *feasible* (avoids walls/obstacles) by combining the cost-so-far (g) with a heuristic (typically Manhattan distance) to the goal.

- **Dynamic Replanning:**

Since multiple agents are navigating simultaneously, the environment is dynamic — i.e., other agents may occupy or block positions on the grid. To prevent collisions, the A* path is **recomputed at every timestep**, treating other agents as temporary obstacles. This ensures the path stays valid even if others move unpredictably.

```
def a_star(grid, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic(start, goal), 0, start))
    came_from = {}
    g_score = {start: 0}

    while open_list:
        _, g, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path

        for dx, dy in DIRECTIONS.values():
            next_pos = (current[0] + dx, current[1] + dy)

            if not valid_move(grid, next_pos[0], next_pos[1]):
                continue

            tentative_g = g + 1

            if next_pos not in g_score or tentative_g < g_score[next_pos]:
                g_score[next_pos] = tentative_g
```

```

        f_score = tentative_g + heuristic(next_pos, goal)
        heapq.heappush(open_list, (f_score, tentative_g, next_pos))
        came_from[next_pos] = current

    return None

```

```

def find_nearest_free_agent(agents_info, target_pos, agent_busy):
    min_dist = float('inf')
    nearest_idx = None
    for idx, agent in enumerate(agents_info):
        if agent_busy[idx]: # skip busy agents
            continue
        agent_pos = tuple(agent['position'])
        dist = abs(agent_pos[0] - target_pos[0]) + abs(agent_pos[1] - target_pos[1])
        if dist < min_dist:
            min_dist = dist
            nearest_idx = idx
    return nearest_idx

```

4.2 Centralized Training with Decentralized Execution

This implementation follows a Centralized Training with Decentralized Execution (CTDE) paradigm with implicit task allocation in a Multi-Agent Deep Deterministic Policy Gradient (MADDPG) setup.

Centralized Critic: During training, each agent uses a centralized critic that takes in the global state and joint actions of all agents. This provides better gradient estimates since the critic has full observability of the environment and all agents' actions.

Decentralized Actor: During execution (i.e., when interacting with the environment), each agent selects actions based solely on its own local observation using its actor network. This matches the decentralized constraints of many real-world multi-agent systems.

Implicit Task Allocation: Task allocation is not hardcoded or explicitly defined. Instead, agents learn to coordinate and divide tasks implicitly through shared training:

- The critic penalizes or rewards agents based on the joint outcome.
- Each agent's actor learns to optimize its own behavior in a way that complements others to improve overall performance.

Over time, agents specialize based on their local observations and gradients propagated from the centralized critic.

4.2.1 Code Implementation

```

class Agent:
    def __init__(self, actor_dims, critic_dims, n_actions, n_agents, agent_idx, chkpt_dir,
                 alpha=0.01, beta=0.01, fc1=128,
                 fc2=64, gamma=0.99, tau=0.01):
        self.gamma = gamma
        self.tau = tau
        self.n_actions = n_actions
        self.agent_name = 'agent_%s' % agent_idx
        self.actor = ActorNetwork(alpha, actor_dims, fc1, fc2, n_actions,

```

```

        chkpt_dir=chkpt_dir, name=self.agent_name+'_actor')
self.critic = CriticNetwork(beta, critic_dims,
                             fc1, fc2, n_agents, n_actions,
                             chkpt_dir=chkpt_dir, name=self.agent_name+'_critic')
self.target_actor = ActorNetwork(alpha, actor_dims, fc1, fc2, n_actions,
                                  chkpt_dir=chkpt_dir,
                                  name=self.agent_name+'_target_actor')
self.target_critic = CriticNetwork(beta, critic_dims,
                                    fc1, fc2, n_agents, n_actions,
                                    chkpt_dir=chkpt_dir,
                                    name=self.agent_name+'_target_critic')

self.update_network_parameters(tau=1)

def choose_action(self, observation, epsilon=0.2, evaluate=False):
    if evaluate:
        epsilon = 0.0
    state = T.tensor([observation], dtype=T.float).to(self.actor.device)
    action_probs = self.actor.forward(state)

    # Epsilon-greedy exploration
    if np.random.random() < epsilon:
        action = np.random.randint(0, self.n_actions)
    else:
        action = T.argmax(action_probs, dim=1).item()

    return action

def update_network_parameters(self, tau=None):
    if tau is None:
        tau = self.tau

    target_actor_params = self.target_actor.named_parameters()
    actor_params = self.actor.named_parameters()

    target_actor_state_dict = dict(target_actor_params)
    actor_state_dict = dict(actor_params)
    for name in actor_state_dict:
        actor_state_dict[name] = tau*actor_state_dict[name].clone() + \
            (1-tau)*target_actor_state_dict[name].clone()

    self.target_actor.load_state_dict(actor_state_dict)

    target_critic_params = self.target_critic.named_parameters()
    critic_params = self.critic.named_parameters()

    target_critic_state_dict = dict(target_critic_params)
    critic_state_dict = dict(critic_params)
    for name in critic_state_dict:
        critic_state_dict[name] = tau*critic_state_dict[name].clone() + \
            (1-tau)*target_critic_state_dict[name].clone()

    self.target_critic.load_state_dict(critic_state_dict)

def save_models(self, ):
    self.actor.save_checkpoint()
    self.target_actor.save_checkpoint()
    self.critic.save_checkpoint()

```

```

        self.target_critic.save_checkpoint()

def load_models(self):
    self.actor.load_checkpoint()
    self.target_actor.load_checkpoint()
    self.critic.load_checkpoint()
    self.target_critic.load_checkpoint()

```

```

class MADDPG:
    def __init__(self, actor_dims, critic_dims, n_agents, n_actions,
                  scenario='simple', alpha=0.01, beta=0.01, fc1=128,
                  fc2=64, gamma=0.99, tau=0.01, batch_size=128, chkpt_dir='tmp/maddpg/'):
        self.agents = []
        self.n_agents = n_agents
        self.n_actions = n_actions
        chkpt_dir += scenario
        for agent_idx in range(self.n_agents):
            self.agents.append(Agent(actor_dims[agent_idx], critic_dims,
                                     n_actions, n_agents, agent_idx, alpha=alpha, beta=beta, fc1=fc1, fc2=fc2,
                                     ↪ gamma=gamma, tau=tau,
                                     chkpt_dir=chkpt_dir))
        self.memory = MultiAgentReplayBuffer(10000, critic_dims, actor_dims,
                                             n_actions, n_agents, batch_size)
        self.batch_size = batch_size

    def learn(self):
        T.set_default_dtype(T.float32)
        if not self.memory.ready():
            return

        actor_states, states, actions, rewards, \
        actor_new_states, states_, dones = self.memory.sample_buffer()

        device = self.agents[0].actor.device

        states = T.tensor(states, dtype=T.float).to(device)
        actions_tensor = T.tensor(actions, dtype=T.float).to(device)
        rewards = T.tensor(rewards).to(device)
        states_ = T.tensor(states_, dtype=T.float).to(device)
        dones = T.tensor(dones).to(device)

        all_agents_new_actions = []
        all_agents_new_mu_actions = []
        old_agents_actions = []

        for agent_idx, agent in enumerate(self.agents):
            # Process new states to get target policy output
            new_states = T.tensor(actor_new_states[agent_idx],
                                  dtype=T.float).to(device)

            # Get action probabilities from target actor
            new_pi = agent.target_actor.forward(new_states)
            # Convert to discrete actions
            new_actions = T.argmax(new_pi, dim=1).float().view(-1, 1)
            all_agents_new_actions.append(new_actions)

            # Process current states to get policy output

```

```

mu_states = T.tensor(actor_states[agent_idx],
                      dtype=T.float).to(device)
pi = agent.actor.forward(mu_states)
# Convert to discrete actions
current_actions = T.argmax(pi, dim=1).float().view(-1, 1)
all_agents_new_mu_actions.append(current_actions)

# Get stored actions
old_actions = actions_tensor[agent_idx].view(-1, 1)
old_agents_actions.append(old_actions)

# Concatenate discrete actions for critic input
new_actions = T.cat([acts for acts in all_agents_new_actions], dim=1)
mu = T.cat([acts for acts in all_agents_new_mu_actions], dim=1)
old_actions = T.cat([acts for acts in old_agents_actions], dim=1)

for agent_idx, agent in enumerate(self.agents):
    # Update critic loss using joint actions
    critic_value_ = agent.target_critic.forward(states_, new_actions).flatten()
    critic_value_[done[:,0]] = 0.0
    critic_value = agent.critic.forward(states, old_actions).flatten() # to float

    target = (rewards[:, agent_idx] + agent.gamma * critic_value_).float()
    critic_loss = F.mse_loss(target, critic_value)
    agent.critic.optimizer.zero_grad()
    critic_loss.backward()
    agent.critic.optimizer.step()

    # Update actor loss
    actor_loss = agent.critic.forward(states, mu).flatten()
    actor_loss = -T.mean(actor_loss)
    agent.actor.optimizer.zero_grad()
    actor_loss.backward()
    agent.actor.optimizer.step()

    agent.update_network_parameters()

```

4.2.2 Shortcomings

While implementing the CTDE-based MADDPG framework, we encountered several key limitations that hindered the performance and practicality of the approach:

- **Implicit Task Allocation Issues:** We observed that relying solely on implicit task allocation through shared gradients and rewards did not consistently lead to complementary agent behaviors. In many cases, the agents developed redundant strategies or interfered with each other, especially in tasks that required clear division of labor.
- **Extensive Hyperparameter Tuning:** The algorithm was highly sensitive to hyperparameters such as learning rates, soft target update coefficients, and exploration noise. Finding a stable and performant set of parameters required exhaustive tuning, which became increasingly difficult as the number of agents increased.
- **Reward Shaping Challenges:** Designing an effective reward function proved to be a major bottleneck. Without explicit task definitions, shaping the reward to encourage cooperative and coordinated behavior was non-trivial. Poorly defined rewards often led to suboptimal or uncoordinated policies.

- **High Computational Requirements:** Training centralized critics for all agents imposed a significant computational burden. Due to limited computational resources, we faced slow training cycles, and replay buffer management became a challenge, particularly when trying to balance data diversity with memory constraints.
- **Complex and Dynamic Environment:** The environment we used involved intricate dynamics and frequent interactions among agents. This made the joint action space highly non-stationary, complicating the learning process. The centralized critic struggled to generalize across such a diverse state-action distribution, especially under limited rollout data.

4.3 Centralized PPO

Proximal Policy Optimization (PPO) is a policy gradient algorithm that updates the policy in a stable and constrained manner by clipping the policy objective. In the centralized setting, PPO is extended to handle multi-agent systems by using a single centralized policy and value network that operate on the joint observation and action spaces of all agents.

Centralized Policy: Instead of having separate policies for each agent, a single centralized policy $\pi_\theta(a_1, a_2, \dots, a_N | s)$ is learned. This policy takes the full state s (which includes the observations of all agents) as input and outputs a joint action $(a_1, a_2, \dots, a_N)$ for all N agents. This allows the policy to directly coordinate the actions of multiple agents.

Centralized Critic: A single value function $V_\phi(s)$ is also trained using the full global state s . This critic estimates the expected return from the joint state, providing richer feedback for policy updates.

Training Objective: The standard PPO clipped surrogate objective is used for updating the policy:

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio and $\hat{A}_t$ is the advantage estimate at time t .

Value Function Loss: The value network is trained to minimize the squared error between predicted and target returns:

$$L^{VF}(\phi) = E_t \left[(V_\phi(s_t) - \hat{R}_t)^2 \right]$$

Advantage Estimation: Advantages are typically estimated using Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l} \quad \text{where} \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Algorithm Overview:

1. Collect trajectories by running the centralized policy in the environment, storing (s_t, a_t, r_t, s_{t+1}) .
2. Estimate advantages $\hat{A}_t$ and returns $\hat{R}_t$ using GAE.
3. Update policy network using PPO's clipped objective.
4. Update value network using mean squared error loss.
5. Repeat for multiple epochs with mini-batches to improve sample efficiency.

Execution: During execution, the centralized policy receives the global state and outputs a joint action vector for all agents, which are then executed in the environment. This approach assumes full observability and central control, which may be available in simulation or with a centralized controller in real-world applications.

4.3.1 Shortcomings

Through our implementation and evaluation of Centralized PPO, we identified several limitations that made it less suitable for our multi-agent setup:

- **Single Point of Control:** In our approach, a single centralized policy was used to output the actions for all agents. This introduced significant latency and computational overhead, making it challenging to scale the solution or apply it in scenarios requiring fast or independent agent responses.
- **Scalability Issues:** As we increased the number of agents, we noticed a sharp rise in the size of the joint observation and action spaces. This made the policy harder to train and less sample efficient, especially in our resource-constrained setting.
- **Limited Generalization:** The centralized policy was tightly coupled to the specific agent configuration used during training. Any modification, such as adding or removing agents, required us to retrain the entire policy, limiting flexibility and adaptability.
- **Single Point of Failure:** We also observed that any instability or failure in the centralized policy immediately affected all agents. Since every agent relied on the same output, even a minor policy degradation could compromise the behavior of the entire system.

4.4 Hierarchical Reinforcement Learning (HRL)

To better manage complexity and improve coordination, we explored Hierarchical Reinforcement Learning (HRL) in our multi-agent setup. The idea was to introduce a two-level control structure: a high-level policy that selects options (e.g., move toward a request or destination), and a low-level policy that executes primitive actions based on the chosen option and current observations. Ideally, HRL should allow the agent to learn to solve the environment faster, given that its goal becomes simpler and broken down into subtasks/options. Using HRL also improves explainability. However, with our implementation, we were unable to get the agent to learn effectively.

High-Level Policy: We used a DQN-based approach where the high-level policy selected one of several discrete options. These options included directives like "move toward request i " (chosen from active requests) or "move toward destination d " (from a set of possible destinations). This policy operated at a lower temporal frequency, allowing each selected option to be followed for a fixed duration before re-evaluating.

Low-Level Policy: The low-level policy was trained to output primitive actions that guide the agent toward the target specified by the current option. This policy took in the agent's local observations and the selected high-level directive, allowing for modular and reusable low-level behaviors.

4.4.1 Shortcomings

While HRL provided a natural structure for decomposing tasks, our implementation wasn't able to learn well. We speculate that these are the main shortcomings

- **Option Granularity:** Designing meaningful and effective options was non-trivial. The predefined options like "move toward request i " often failed to capture context-sensitive behaviors or prioritize between competing goals, reducing the effectiveness of the hierarchy.
- **Training Complexity:** Training both high-level and low-level policies introduced additional complexity. Coordination between the two levels required careful tuning, and misalignment between them often led to suboptimal or conflicting behavior.
- **Sparse and Delayed Feedback:** The high-level policy received feedback only after an option was executed for several steps, making learning slow and unstable. This delayed reward signal made it hard to evaluate the impact of a high-level decision accurately.

- **Lack of Adaptivity:** Once an option was selected, agents committed to it for a fixed period. This rigid structure hindered quick adaptation to dynamic changes in the environment, especially in highly interactive or fast-paced settings.
- **Coordination Overhead:** In multi-agent settings, it was difficult to ensure consistent coordination among agents solely through high-level options. Independent option selection by agents sometimes led to conflicting or inefficient group behaviors.

5 Evaluation Metrics

To assess the performance and practicality of our intelligent warehouse system, we employ several metrics rooted in mechanical systems efficiency, control, and logistics. These metrics are categorized into task performance, path efficiency, safety, and system-level operational quality.

5.1 Task and Operational Efficiency

- **Throughput Rate:**

$$\text{Throughput} = \frac{N_{\text{completed tasks}}}{T_{\text{total}}}$$

Measures the number of successful package deliveries per unit time. It reflects overall warehouse productivity.

- A* Algorithm (Nearest Task): 0.045
- Multi-Agent CTDE RL: **0.041** (slightly lower due to exploration overheads and coordination complexity)

- **Utilization Rate of AGVs:**

$$\text{Utilization} = \frac{T_{\text{active}}}{T_{\text{total}}}$$

The proportion of time an AGV is actively engaged in meaningful tasks. Low utilization indicates suboptimal scheduling or idle vehicles.

- A* Algorithm: 34.34%
- Multi-Agent CTDE RL: **40.84%** (better load distribution and task scheduling via learned policies)

- **Average Task Completion Time:**

$$\bar{T}_{\text{task}} = \frac{1}{N} \sum_{i=1}^N T_i$$

Represents the average time taken to complete each task, from pickup to delivery.

- A* Algorithm: 64.70 steps
- Multi-Agent CTDE RL: **69.10 steps** (worse performance but lower collisions and distance)

5.2 Path Planning and Motion Efficiency

- **Total Distance Traveled by AGVs:**

$$D_{\text{total}} = \sum_{i=1}^N d_i$$

The cumulative distance moved by all agents. Lower values generally indicate more efficient routing.

- A* Algorithm: 336.52 units
- Multi-Agent CTDE RL: **325.87 units** (coordination reduces unnecessary movement)

5.3 Safety and Collision Avoidance

- **Collision Count:**

$$C = \sum_{t=1}^T 1_{\{\text{collision at time } t\}}$$

Tracks the number of collision events. High collision rates indicate poor motion coordination and unsafe policies.

- A* Algorithm: 13.35
- Multi-Agent CTDE RL: **6.7** (coordination-aware policy reduces collisions significantly)

6 Future Improvements

Throughout the course of this project, we encountered several challenges that opened up promising directions for further refinement. Based on our observations and preliminary experiments, we outline the following improvements that we intend to pursue:

- **Improved Reward Design:** One key challenge we identified was designing a reward function that effectively encourages coordination among agents. Initial formulations often led to suboptimal or conflicting behaviors. In future iterations, we plan to further refine the reward structure to better reflect intermediate sub-goals and individual contributions, thereby promoting more robust and cooperative strategies.
- **Improving HRL:** Ironing out the issues with the HRL implementation should prove to give the best results and solve the environment faster.
- **Curriculum Learning:** To enhance training efficiency in complex environments, we aim to integrate a more structured curriculum learning approach. By gradually increasing the difficulty of tasks, agents can first acquire fundamental skills before tackling harder scenarios. This staged learning is expected to improve exploration, stability, and convergence speed.
- **Encouraging Cooperation:** We intend to introduce explicit mechanisms to foster inter-agent collaboration. This includes enabling communication between agents, assigning specialized roles (such as picker, carrier, etc.), designing shared reward structures to align goals, and exploring zone-based or territory-based specialization. These additions are expected to enhance collective performance and role-aware behavior.
- **Hybrid Approaches:** Rather than relying entirely on reinforcement learning, we aim to explore hybrid strategies that combine learned policies with rule-based guidance. In certain well-understood states or situations, using predefined rules can offer more reliability and safety, while RL handles decision-making in more complex or uncertain regions. This blend can help reduce training time, improve generalization, and ensure better performance in safety-critical scenarios.